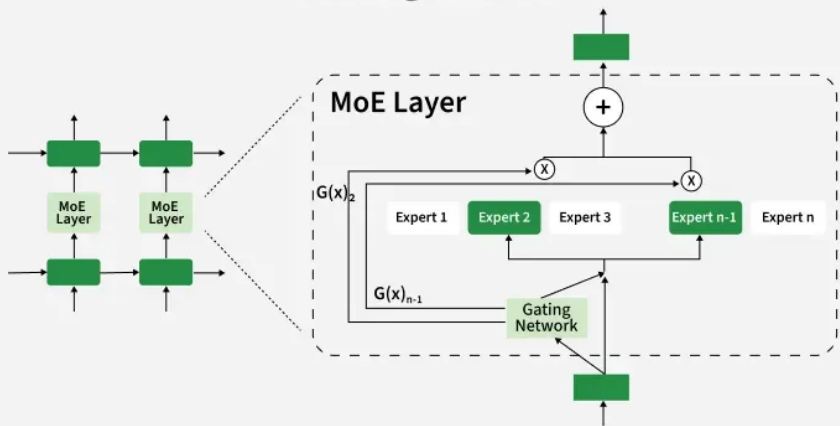


Mixture of Experts Explained

Working of MOE



! THE PROBLEM

Scaling laws said "bigger is better." Compute said "no."

In a standard ("dense") transformer, **every parameter activates for every token**. Double the parameters, and you double the compute cost per token.

DENSE MODEL

70B params → all 70B used per token → expensive, slow

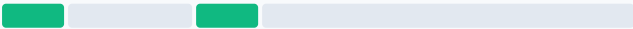
Want 10× the capacity? → 10× the compute per token. Not viable.

The core question: Does every token really need the *entire* model? Predicting a closing parenthesis vs translating French are wildly different tasks. What if a model learned to use only the relevant portion of itself?

DENSE (capacity = compute)

token →  all fire

MoE (capacity ≠ compute)

token →  few fire



AI Coach John



PROITBRIDGE
Strive For Better Future

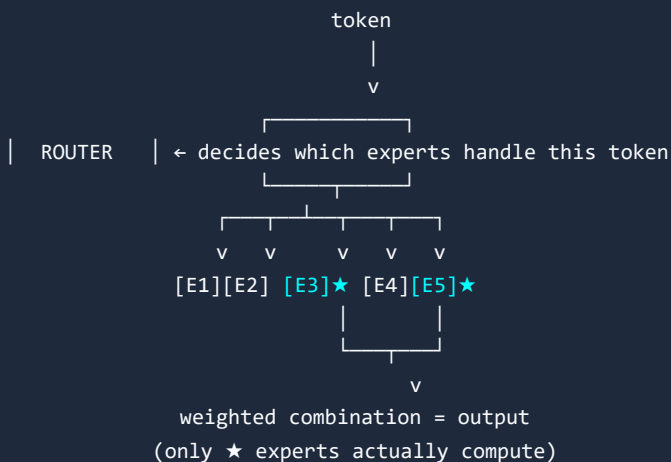
Follow

↻ Repost

💡 THE CORE IDEA

Sparse Activation: Many experts, only a few used

An MoE layer replaces one big feedforward network with **many smaller ones** (experts), plus a **router** that picks which few to use for each token.



- **Total parameters:** determines capacity and memory footprint.
- **Active parameters:** what actually computes per token (speed).



AI Coach John



Follow

↻ Repost

Total vs Active Parameters: The number that matters

MIXTRAL-STYLE 8x7B MoE

Total params: ~47B ← what you must load into memory

Active params: ~13B ← what computes per token

QWEN3-CODER 30B MoE

Total params: 30B

Active params: ~3.3B ← only ~11% fires per token

The crucial caveat: You still need memory for **all** parameters, even the inactive ones. Every expert must be resident because the router might need them instantly.

THE BIGGEST MISCONCEPTION

MEMORY needed:  ALL params (47B)

COMPUTE per tok:  ACTIVE (13B)

⚠ MoE saves compute, NOT memory



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

How the Router Works: The small network deciding all

The router is a lightweight learned layer that scores each expert for the incoming token, then selects the top-k highest scorers.

```
# Conceptual shape of top-2 routing
logits = router_layer(token_hidden_state) # 1 score per expert
probs = softmax(logits)                  # e.g. [0.02, 0.51, 0.31, 0.09]

top_k_experts = top_k(probs, k=2)        # pick experts 2 and 3
output = sum(prob_i * expert_i(token) for i in top_k_experts)
```

Critically: routing happens **per token**, not per prompt. Different words in the same sentence can go to entirely different experts.

EACH TOKEN ROUTED INDEPENDENTLY

```
"def calc_sum (a, b):"
  ↓      ↓      ↓      ↓
E3,E7   E1,E3   E3,E5   E2,E7
```



AI Coach John



Follow

 Repost

What Do Experts Specialize In? **Not what you assume**

The common assumption: one expert learns math, another learns French, another learns code. A clean, human-readable division of labor.

The reality: specialization is usually subtle and syntactic (e.g., punctuation patterns, numeric formatting). The router just learns whatever partitioning reduces loss.

✗ EXPECTED

Math

Code

Lang

Clean domains

✓ OBSERVED

??

??

??

Emergent, subtle, syntactic

Experts are NOT modular plugins you can swap.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↺ Repost

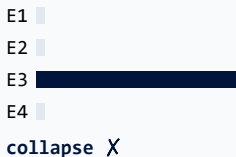
The Load Balancing Problem: Left alone, the router plays favorites

THE COLLAPSE LOOP:

1. E3 gets picked slightly more early on
2. E3 gets more training signal
3. E3 gets better
4. Router picks E3 even more
5. Other experts starve → "expert collapse"

The fix: An auxiliary load balancing loss is added during training that penalizes uneven expert usage.

WITHOUT BALANCING



WITH BALANCING



AI Coach John



Follow

 Repost

Expert Capacity: When too many tokens want the same expert

GPUs need fixed-size tensor shapes. During training, experts are given a **capacity factor**—a strict cap on tokens they can process per batch.

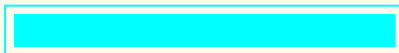
Expert 3 capacity: 128 tokens

Tokens routed to Expert 3: 145

└─> 17 tokens exceed capacity
→ dropped (passed through via residual connection)

Dropped tokens don't break the model; they just skip this MoE layer's FFN transformation. Quality degrades gracefully, not catastrophically.

EXPERT 3 (capacity: 128)



128 processed ✓

17 tokens overflow → skipped via residual ⚠



AI Coach John



Follow

↻ Repost

Where MoE Sits: It replaces the FFN, not attention



Why the FFN? It holds $\sim 2/3$ of a transformer's parameters, so it's where sparsity pays off most. It's also a per-token transformation with no cross-token dependency.

Why not attention? Every token genuinely needs to attend to every other token. Routing a token to only "some" attention would blind it to parts of the context.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

MoE Tradeoffs: What you gain and pay

THE GAINS

- ✓ Far more capacity per FLOP (better quality at same inference cost)
- ✓ Faster inference than a dense model of equivalent total size
- ✓ Cheaper to train to a given quality level
- ✓ Scales capacity without linearly scaling per-token compute

THE COSTS

- ✗ Full memory footprint required for all experts
- ✗ Training instability (routing collapse)
- ✗ Harder to fine-tune without destabilizing routing
- ✗ Deployment complexity (expert parallelism across GPUs)



Dense vs MoE: When each is the right choice

	Dense Model	MoE Model
Params active	All of them	Small fraction (5-25%)
Memory needed	= total params	= total params
Compute / token	High	Much lower
Fine-tuning	Well-understood	Trickier, routing drifts
Examples	Llama 3 70B, Phi-4	Mixtral, DeepSeek-V3



If memory constrained → Dense.

If compute/latency constrained → MoE.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

What It Means For You:

Decisions you actually make

- ✓ **Reading specs:** "30B total / 3.3B active" answers two different questions.
Hardware fit = total. Speed/cost = active.
- ✓ **Local deployment:** An MoE needs the same VRAM as a dense model of its total size. Check the total first, always.
- ✓ **Cost per token:** MoE models are often priced lower than their total size suggests, since providers pay compute on active parameters.
- ✓ **Quantization:** With a large total footprint, quantization is often what makes an MoE runnable locally at all.
- ✓ **Throughput planning:** Theoretical compute savings don't fully translate to real throughput due to uneven expert load.

Model card: "30B total, 3.3B active"



"Will it FIT?"
(needs ~30B memory)



"How FAST?"
(runs like ~3B model)



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

Common Misconceptions: What people get wrong

× MoE uses less memory

All experts must be loaded. MoE saves compute, not VRAM.

× Experts = topic specialists

Specialization is emergent and syntactic, not clean human domains.

× Router picks per prompt

Routing is **per token**. A single sentence touches many experts.

× MoE is always better

For a fixed memory budget, a dense model may outperform an MoE.

× Experts are swappable

Experts are jointly trained and interdependent, not modular plugins.

× Active count = quality

Quality tracks between active and total counts, not equal to either.

